

KAIROS OS: Prototyping with Loop Engineering

1. Executive Summary

This document outlines a strategy to go from concept to an actual prototype of KAIROS OS within the 1-month 'Disconnected by Design' hackathon timeline. To achieve this ambitious goal, we will utilize **Loop Engineering**—a methodology that replaces manual step-by-step AI prompting with automated, self-contained systems (loops) that discover work, execute it, verify it, and persist state. By shifting from 'doing the work' to 'designing the loops', we can safely accelerate development.

2. Applying the Five Moves of Loop Engineering

Every loop we design will implement the five essential moves to ensure reliability and avoid common pitfalls like 'Nodding Loops' (missing verification) and 'Amnesiac Loops' (missing persistence):

- **Discovery:** Loop runs on a timer to read the KAIROS PRD and Tech Spec, surfacing new actionable tickets (e.g., 'Implement Intent Gate UI', 'Setup Composio Gmail Integration').
- **Handoff:** Using `--worktree`, each generated ticket gets an isolated git directory to avoid parallel agents colliding.
- **Verification:** We enforce the *Maker-Checker* principle. The Generator agent writes the code, while a separate Evaluator agent tests it (e.g., runs Next.js API tests, compiles Jetpack Compose previews). The Evaluator assumes the code is **BROKEN** until proven otherwise.
- **Persistence:** Progress is recorded in `./state/hackathon_progress.md` and automated pull requests are raised to GitHub.
- **Scheduling:** Loops will be scheduled via local timers during the day for rapid iteration, and via GitHub Actions for overnight verification runs.

3. The KAIROS Development Loops

Loop A: The Android Client Loop

Builds the thin Kotlin + Jetpack Compose frontend. Triggered by frontend UI tickets. The Evaluator agent uses the Android Lint tool and unit tests to verify the UI. To prevent 'Comprehension Rot', humans will review PRs for the visual aesthetic of the blinking cursor interface and the Floating Bubble overlay.

Loop B: The Next.js Fat Backend Loop

Builds the intent router and MCP integrations. The Generator agent implements the Gemini 3.5 Flash tiered routing and hooks up Composio SDKs. The Evaluator agent writes and runs Jest API tests against the endpoints (e.g., simulating a `@gmail` command request) to ensure JSON widget payloads

are correctly structured.

4. Guardrails & Operational Discipline

To ensure our agentic loops don't spin out of control, we will enforce strict guardrails:

- **Token Caps:** To prevent 'Token Blowout' overnight, we will set strict per-run token limits and a maximum number of retry attempts for the Evaluator agent.
- **Human-in-the-Loop (Keep One Door Open):** We will never auto-merge. To prevent 'Cognitive Surrender', team members will conduct morning triages to review the PRs generated by the loops.
- **Skills as Intent Debt Payoff:** KAIROS OS architecture rules (e.g., 'Always return Widget JSON, never Markdown') will be encoded into SKILL .md files, ensuring the agents always have the correct context without bloating the prompt.

5. Conclusion

By applying Loop Engineering, we can offload the mechanical labor of boilerplate Android setup and Next.js wiring to autonomous systems. This frees our human judgment to focus on what matters most for the hackathon: perfecting the minimalist 'Kairos' philosophy, refining the Intent Gates, and ensuring the final prototype perfectly answers the question: *'What does it mean to be human?'*